

Test-Time Compute: Razonamiento Extendido

Módulo 8 · Ingeniería de Modelos y LLMOps

o1/o3 vs GPT-4o, extended thinking, Best-of-N, routing dinámico y calibración del budget de razonamiento.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Test-Time Compute: Razonamiento Extendido

Hasta 2024, la forma dominante de mejorar el rendimiento de los LLMs era escalar el preentrenamiento: más parámetros, más datos, más cómputo de entrenamiento. En 2024-2025, emergió un paradigma complementario: en lugar de invertir todo el cómputo en el entrenamiento, invertir más cómputo en el tiempo de inferencia para producir respuestas de mayor calidad. Este paradigma se llama test-time compute (TTC) o inference-time scaling.

Los modelos de razonamiento como o1, o3 y o4-mini de OpenAI, y Claude con extended thinking, implementan TTC de forma nativa: internamente generan largas cadenas de "pensamiento" antes de producir la respuesta final. El resultado es un rendimiento muy superior en tareas de razonamiento complejo, matemáticas, código y lógica, a cambio de mayor latencia y mayor coste por consulta. Para los equipos que diseñan sistemas LLM, entender qué es TTC, cuándo usarlo, y cómo calibrar el balance coste/calidad es una competencia técnica crítica.

Los mecanismos de test-time compute

Chain of Thought como forma básica de TTC

El CoT explícito (instruir al modelo a razonar paso a paso) es la forma más básica de TTC: el modelo usa tokens adicionales de salida para "pensar" antes de dar la respuesta final. Este pensamiento visible no es solo para el usuario: guía la generación de los tokens posteriores, mejorando la calidad de la respuesta. El CoT implícito (como en o1/o3) realiza el mismo proceso pero internamente, sin mostrar el razonamiento al usuario.

Extended thinking en modelos modernos

Los modelos de razonamiento (o1, o3, o4-mini de OpenAI; Claude con extended thinking; Gemini 2.0 Flash Thinking) realizan un proceso de pensamiento interno antes de generar la respuesta. El modelo puede explorar múltiples caminos de razonamiento, detectar inconsistencias en su propio razonamiento, y corregirse a sí mismo, todo dentro del proceso de generación. El pensamiento interno no se cobra al usuario en todos los modelos (varía por proveedor y configuración).

Best-of-N sampling: elegir la mejor de múltiples respuestas

Best-of-N (BoN) genera N respuestas independientes y selecciona la mejor según un criterio de evaluación. Con N=8 y un verifier (modelo más pequeño entrenado para evaluar la corrección de las respuestas), BoN puede alcanzar el rendimiento de modelos mucho más grandes a un coste inferior. DeepMind demostró que BoN puede ser más eficiente que escalar el modelo en muchos benchmarks de razonamiento matemático.

Beam search y búsqueda en árbol

Las técnicas de búsqueda más sofisticadas (Monte Carlo Tree Search, MCTS, aplicado a LLMs) exploran sistemáticamente el árbol de posibles cadenas de razonamiento, evaluando cada nodo con un Process Reward Model (PRM) que estima la calidad del razonamiento en cada paso (en

lugar de solo al final). Estas técnicas producen mejoras significativas en tareas donde el razonamiento multi-paso es crítico (matemáticas de competición, programación compleja, planificación).

SECCIÓN 3 · CÓMO FUNCIONA

Cuándo usar modelos de razonamiento vs modelos estándar

Los modelos de razonamiento (o1, o3, Claude con extended thinking) son superiores en: problemas matemáticos multi-paso, verificación de código complejo, razonamiento lógico con múltiples restricciones, planificación estratégica con incertidumbre, y cualquier tarea donde el error de un paso intermedio invalida la solución completa. Son inferiores (o iguales a un coste mayor) en: generación de texto creativo, clasificación simple, extracción de información directa, y conversación estándar.

La métrica clave para decidir si usar TTC es la "hardness" del problema: qué fracción de los modelos estándar falla en ese tipo de problema. Si GPT-4o ya tiene >90% de éxito en tu tipo de tarea con CoT, un modelo de razonamiento no añadirá valor significativo. Si GPT-4o tiene <70% de éxito con CoT, el razonamiento extendido puede mejorar significativamente.

Budget de tokens de razonamiento

Los modelos de razonamiento tienen parámetros para controlar cuántos tokens de pensamiento usar. Claude con extended thinking: thinking budget en tokens (mayor budget = más razonamiento = mejor calidad en tareas difíciles pero mayor coste y latencia). OpenAI o3/o4-mini: niveles de esfuerzo (low, medium, high). Para sistemas de producción, calibrar el budget de razonamiento según la dificultad estimada de cada consulta (routing dinámico) puede optimizar el balance coste/calidad.

SECCIÓN 4 · EJEMPLOS REALES

- o3 en resolución de problemas matemáticos (empresa de finanzas cuantitativas): los analistas usan o3 para verificar derivaciones de fórmulas de pricing de opciones complejas. o3 con extended thinking encuentra errores en demostraciones matemáticas que GPT-4o perdía consistentemente. El mayor coste (5-10x por consulta) es justificable por la criticidad de la corrección.
- Routing dinámico (empresa de IA): el sistema clasifica primero la dificultad de cada consulta (fácil/media/difícil) con un modelo pequeño. Las consultas fáciles van a GPT-4o-mini (barato, rápido). Las medias van a GPT-4o. Las difíciles van a o3 o Claude con extended thinking. Este routing reduce el coste total un 40% manteniendo la calidad en las consultas difíciles.
- Best-of-N para generación de código crítico: para código de algoritmos financieros donde la corrección es crítica, se generan N=8 implementaciones con GPT-4o (en paralelo), se ejecutan los tests automatizados en cada una, y se selecciona la que pasa todos los tests. La probabilidad de obtener al menos una implementación correcta entre 8 es mucho mayor que con una sola generación.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Usar modelos de razonamiento para todas las tareas. o3 y o1 tienen latencia 5-30x mayor y coste 5-20x mayor que GPT-4o. Para clasificación, extracción, y generación estándar, son overkill. El routing adecuado según dificultad del problema puede reducir el coste total un 40-60% sin sacrificar calidad en las tareas difíciles.

Error 2: Confundir el razonamiento visible con el razonamiento correcto garantizado. Los modelos de razonamiento pueden producir cadenas de razonamiento largas que suenan convincentes pero son incorrectas. El razonamiento extendido reduce la tasa de errores pero no la elimina. Para aplicaciones críticas (cálculos financieros, código de producción), sigue siendo necesaria la verificación externa.

Error 3: Ignorar el efecto del budget de razonamiento en la calidad. Con un budget demasiado pequeño, el modelo de razonamiento puede truncar su pensamiento y producir una respuesta incorrecta. Con un budget demasiado grande, el coste y la latencia crecen sin mejora proporcional de calidad. Calibra el budget sobre tu golden dataset de tareas difíciles.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

Guía de selección de modelo para producción basada en complejidad de tarea: tareas simples (clasificación, extracción directa, QA con contexto) → GPT-4o-mini, Claude Haiku, o modelo open source pequeño (7-8B). Tareas de complejidad media (análisis, síntesis, código de nivel medio) → GPT-4o, Claude Sonnet. Tareas difíciles (matemáticas complejas, código crítico, razonamiento multi-hop) → o3, o4-mini, Claude con extended thinking. El routing dinámico entre niveles es la estrategia óptima para optimizar coste/calidad en sistemas de uso general.

Para evaluar si un modelo de razonamiento mejora sobre uno estándar en tu caso de uso: construye un golden dataset de las 30-50 consultas más difíciles que tu sistema maneja (aquellas donde los usuarios reportan más errores). Evalúa GPT-4o y o3 sobre ese dataset. Si la diferencia de calidad es significativa y el caso de uso la justifica, implementa routing para enviar ese tipo de consultas al modelo de razonamiento.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M6-T2 (CoT): CoT es la forma más básica y accesible de TTC a través de prompting.
- M6-T4 (Self-consistency): Best-of-N es una generalización de self-consistency aplicada a TTC.
- M5-T4 (In-context learning): los modelos de razonamiento combinan ICL con TTC para máximo rendimiento.

SECCIÓN 8 · RESUMEN EJECUTIVO

Test-time compute (TTC) invierte más cómputo en inferencia para mejorar la calidad de respuestas en tareas difíciles. Los modelos de razonamiento (o1/o3/o4-mini, Claude extended thinking) implementan TTC internamente. Las técnicas accesibles son: CoT explícito (básico), Best-of-N (paralelo, selección por verifier), y beam search con PRM. TTC es superior en: matemáticas, código complejo, razonamiento multi-step. No es superior en: tareas simples. El routing dinámico (clasificar dificultad → asignar modelo

apropiado) reduce el coste total un 40-60%. Calibra el budget de razonamiento en tu golden dataset de tareas difíciles.